The Entropy Budget of Multiplication: Duplication, Erasure, and the Landauer Floor in the Two-Move Arithmetic

Working paper v0.1, Project secret3, August 11, 2026

Abstract

The project's linearity paper identified rule R2 as the single erasing clause of the transparent arithmetic and rule R1 as its single duplicating clause. This paper weighs them. Two counters over reduction sequences, dupMass and eraseMass, charge each R1 firing the size of the payload it copies and each R2 firing the size of the payload it drops; both are compiled, together with their congruence and concatenation laws, mirroring the module's step-length family. The budget theorems are exact and quantified over all factors: the canonical run of the product box on m and k duplicates m copies of the numeral of k (boxDup), erases that numeral exactly once (boxErase), and addition is fully linear, duplicating and erasing nothing (sumDup, sumErase); the size of a numeral is 5n plus 4 cells (tSizeNum, with the successor's compiled size of five). Read together with the module's step-count formula, the budgets expose a disagreement of prices: putting the smaller factor first minimizes steps, but putting the larger factor first minimizes erasure, so the time-optimal operand order and the heat-optimal operand order are opposite whenever the factors differ. Computing two times one hundred takes 205 steps and erases 504 cells; one hundred times two takes 401 steps and erases 14. Under Landauer's principle, by which only logically irreversible operations carry a thermodynamic floor, the erasure ledger is the machine's minimum heat bill up to a per-cell constant, and duplication, being reversible in principle, is not. The fast order is the hot order. Scope, the payload accounting convention, and the unary-against-positional caveat are stated exactly.

1. Introduction

Two earlier results set this paper's table. The linearity paper proved, by the occurrence ledger, that in the four-rule arithmetic exactly one clause erases, R2, the multiplication-by-zero rule that discards its argument, and exactly one clause duplicates, R1, the successor rule of multiplication that writes its argument twice; R3 and R4, the addition rules, conserve every payload symbol. And the tree papers compiled the step-count formula for the canonical multiplication run, rLen of boxReduces m k equals m times (k plus 2) plus one, the asymmetric cost that makes the schoolteacher's rule, put the smaller number first, a theorem about speed. What has been missing is the other ledger: not how many steps, but how much material each run copies and how much it destroys. Lines N4 and P1 of the research agenda ask for exactly this, the weights and their physical reading, and this paper delivers both in one motion, because the theorems turn out to be short and the reading turns out to be surprising.

2. The two counters

The step relation StepR carries its payloads explicitly: RuleR1 u v rewrites the box of the successor of u against v into an addition whose right side contains v twice; RuleR2 v rewrites the box of zero against v into the zero numeral, and v is gone; RuleR3 and RuleR4 move and finish additions, touching every payload exactly once. The counters read the payloads off the constructors:

```
stepDup   (RuleR1 u v) = tSize v      -- one fresh copy of v
stepErase (RuleR2 v)   = tSize v      -- one v destroyed
```

with all other clauses zero and the two context constructors passing through; dupMass and eraseMass fold the counters over a reduction sequence. One convention, stated plainly: the counters weigh payloads, the u and v that the rules carry, and not the fixed rule skeletons, the constant handful of cells that each clause's left and right sides add or shed around them. Payload accounting is the right currency for the linearity question, since the skeletons are the same for every input and vanish into additive constants; the module's mass lemmas for the conserving fragment account for skeletons separately.

3. The budget theorems

Four theorems and a size lemma, all compiled, all total, all quantified over every m, k, and payload:

```
sumDup   : dupMass   (sumSteps k x)    = 0
sumErase : eraseMass (sumSteps k x)    = 0
boxDup   : dupMass   (boxReduces m k)  = m * tSize (num k)
boxErase : eraseMass (boxReduces m k)  = tSize (num k)
tSizeNum : tSize (num n)               = n * tSize sb + 4
```

The proofs mirror the module's step-length family lemma for lemma: concatenation laws (dupTrans, eraseTrans), right-context congruence (dupAppRs, eraseAppRs), transport (dupRepl, eraseRepl), then structural induction along the very recursions that build sumSteps and boxReduces. The successor's compiled size is five cells, so a numeral costs 5n plus 4, and the budgets become concrete:

```
run            steps        duplicated        erased
mul 2 3          11         2 x 19 =   38     19
mul 2 100       205         2 x 504 = 1008    504
mul 100 2       401         100 x 14 = 1400   14
```

The first row is the eleven-step run the project's book displays in full; its ledger now has two more columns.

4. The disagreement of prices

Read the three formulas together. Steps, m(k+2)+1, are minimized by making m, the first factor, the smaller one. Erasure, the size of the numeral of k, is minimized by making k, the second factor, the smaller one. Duplicated mass, m times the numeral of k, sides with steps. So whenever the factors differ, the operand order that is best for time is worst for erasure and the order that is best for erasure is worst for time; there is no order that wins both, and the gap is not marginal: at 2 and 100 the fast order erases thirty-six times more material than the slow one. The disagreement is not an inequality the module proves as such; it is two compiled equalities read side by side, and the paper claims exactly that.

5. The Landauer reading

Landauer's principle says that logical irreversibility has a physical price: erasing one bit of information in an environment at temperature T dissipates at least kT ln 2 of heat, a bound derived in 1961, sharpened since, and confirmed in single-particle experiments. Bennett's complement says the price attaches only to erasure: duplication, and computation generally, can in principle be conducted reversibly, and whatever has not been destroyed can be uncomputed rather than discarded. Mapped onto this machine, with the honest conversion that each erased cell is at least one two-val-

ued distinction and so carries a floor of at least kT ln 2 up to a representation constant: eraseMass is the run's minimum heat bill, and dupMass is not. The conclusion of the previous section then acquires its physical voice. The schoolteacher's rule optimizes a cost that physics does not charge for and pessimizes the one it does; the fast order is the hot order, and a multiplication that must run cool should put its large factor first and take the extra steps. Two caveats keep the reading honest. First, the bound is a floor, not a bill actually paid; real implementations dissipate orders of magnitude above it, and nothing here estimates engineering overheads. Second, this arithmetic is unary on purpose, a microscope rather than a race car: positional numerals would shrink the numeral of k to logarithmic size and the whole budget with it, so the theorems price this machine, not multiplication as such, and the paper says so rather than implying otherwise.

6. What this buys the program

Row A gains the weighted ledger the agenda's N4 asked for, in the same style as the length family and reusable for any future run the module constructs, and P1′ s physical line is opened with a result rather than a promise. Row B gains exactness for two chapters: the Price of Forgetting section of the opening chapter, whose thermodynamic moral, forgetting is the only move that must pay, is now a pair of named theorems with numbers attached; and the planned Price List chapter, which inherits the disagreement of prices as its centerpiece, a truth about cost that no reader will guess in advance and every reader can check against the table above.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 1,797 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification, and the entry point prints the theorem inventory, including boxDup, boxErase, sumDup, sumErase, and tSizeNum.

Appendix B: The entropy budget section (module part P)

```
--------------------------------------------------------------------
-- Tier 2, part P: the entropy budget (N4 weights, P1 reading)
--------------------------------------------------------------------

||| payload duplicated by one rule firing: R1 copies its v
stepDup : StepR t u -> Nat
stepDup (RuleR1 u v) = tSize v
stepDup (RuleR2 v)   = 0
stepDup (RuleR3 u v) = 0
stepDup (RuleR4 v)   = 0
stepDup (RCtxL s)    = stepDup s
stepDup (RCtxR s)    = stepDup s

||| payload erased by one rule firing: R2 drops its v
stepErase : StepR t u -> Nat
stepErase (RuleR1 u v) = 0
stepErase (RuleR2 v)   = tSize v
stepErase (RuleR3 u v) = 0
stepErase (RuleR4 v)   = 0
stepErase (RCtxL s)    = stepErase s
stepErase (RCtxR s)    = stepErase s

dupMass : StepsR t u -> Nat
dupMass RDone        = 0
dupMass (RThen s ss) = stepDup s + dupMass ss
```

```
eraseMass : StepsR t u -> Nat
eraseMass RDone        = 0
eraseMass (RThen s ss) = stepErase s + eraseMass ss

dupTrans : (xs : StepsR a b) -> (ys : StepsR b c) ->
  dupMass (rStepsTrans xs ys) = dupMass xs + dupMass ys
dupTrans RDone ys        = Refl
dupTrans (RThen s xs) ys =
  trans (cong (stepDup s +) (dupTrans xs ys))
        (plusAssociative (stepDup s) (dupMass xs) (dupMass ys))

eraseTrans : (xs : StepsR a b) -> (ys : StepsR b c) ->
  eraseMass (rStepsTrans xs ys) = eraseMass xs + eraseMass ys
eraseTrans RDone ys        = Refl
eraseTrans (RThen s xs) ys =
  trans (cong (stepErase s +) (eraseTrans xs ys))
        (plusAssociative (stepErase s) (eraseMass xs) (eraseMass ys))

dupAppRs : {l, r, r' : Term} -> (ss : StepsR r r') ->
  dupMass (rAppRs {l = l} ss) = dupMass ss
dupAppRs RDone        = Refl
dupAppRs (RThen s ss) = cong (stepDup s +) (dupAppRs {l = l} ss)

eraseAppRs : {l, r, r' : Term} -> (ss : StepsR r r') ->
  eraseMass (rAppRs {l = l} ss) = eraseMass ss
eraseAppRs RDone        = Refl
eraseAppRs (RThen s ss) = cong (stepErase s +) (eraseAppRs {l = l} ss)

dupRepl : {a : Term} -> (e : t = t') -> (d : StepsR a t) ->
  dupMass (replace {p = \w => StepsR a w} e d) = dupMass d
dupRepl Refl d = Refl

eraseRepl : {a : Term} -> (e : t = t') -> (d : StepsR a t) ->
  eraseMass (replace {p = \w => StepsR a w} e d) = eraseMass d
eraseRepl Refl d = Refl

||| addition is linear: it duplicates nothing
sumDup : (k : Nat) -> (x : Term) -> dupMass (sumSteps k x) = 0
sumDup Z     x = Refl
sumDup (S k) x =
  trans (dupAppRs {l = Proofs.sb} (sumSteps k x)) (sumDup k x)

||| and erases nothing
sumErase : (k : Nat) -> (x : Term) -> eraseMass (sumSteps k x) = 0
sumErase Z     x = Refl
sumErase (S k) x =
  trans (eraseAppRs {l = Proofs.sb} (sumSteps k x)) (sumErase k x)

sumDupNum : (k, j : Nat) -> dupMass (sumStepsNum k j) = 0
sumDupNum k j =
  trans (dupRepl (sbPowNum k j) (sumSteps k (num j))) (sumDup k (num j))

sumEraseNum : (k, j : Nat) -> eraseMass (sumStepsNum k j) = 0
sumEraseNum k j =
```

```
  trans (eraseRepl (sbPowNum k j) (sumSteps k (num j))) (sumErase k (num j))

||| the duplication bill of multiplication: m copies of the numeral k
boxDup : (m, k : Nat) ->
  dupMass (boxReduces m k) = m * tSize (Proofs.num k)
boxDup Z     k = Refl
boxDup (S m) k =
  cong (tSize (num k) +) $
  trans (dupTrans (rAppRs (boxReduces m k)) (sumStepsNum k (m * k))) $
  trans (cong2 (+)
          (trans (dupAppRs {l = App CS (App Proofs.skb (Proofs.num k))}
                   (boxReduces m k))
                 (boxDup m k))
          (sumDupNum k (m * k)))
        (plusZeroRightNeutral (m * tSize (Proofs.num k)))

||| the erasure bill of multiplication: one numeral k, exactly once
boxErase : (m, k : Nat) ->
  eraseMass (boxReduces m k) = tSize (Proofs.num k)
boxErase Z     k = plusZeroRightNeutral (tSize (Proofs.num k))
boxErase (S m) k =
  trans (eraseTrans (rAppRs (boxReduces m k)) (sumStepsNum k (m * k))) $
  trans (cong2 (+)
          (trans (eraseAppRs {l = App CS (App Proofs.skb (Proofs.num k))}
                   (boxReduces m k))
                 (boxErase m k))
          (sumEraseNum k (m * k)))
        (plusZeroRightNeutral (tSize (Proofs.num k)))

||| the size of a numeral: n successors on the three-symbol zero
tSizeNum : (n : Nat) ->
  tSize (Proofs.num n) = n * tSize Proofs.sb + 4
tSizeNum Z     = Refl
tSizeNum (S n) =
  trans (cong (tSize Proofs.sb +) (tSizeNum n))
        (plusAssociative (tSize Proofs.sb) (n * tSize Proofs.sb) 4)
```

References

1. R. Landauer, ‘Irreversibility and heat generation in the computing process’, IBM Journal of Research and Development 5 (1961), 183-191.
2. C. H. Bennett, ‘Logical reversibility of computation’, IBM Journal of Research and Development 17 (1973), 525-532.
3. C. H. Bennett, ‘The thermodynamics of computation, a review’, International Journal of Theoretical Physics 21 (1982), 905-940.
4. A. Berut et al., ‘Experimental verification of Landauer’s principle linking information and thermodynamics’, Nature 483 (2012), 187-189.
5. Project secret3, ‘One Rule Forgets: Linearity and the Occurrence Ledger’, working paper, August 2026.
6. Project secret3, ‘Tree Numbers and Transparent Arithmetic’, working papers, August 2026.
7. Project secret3, research agenda v0.6, lines N4 and P1, August 11, 2026.